



Writing Low-Latency C++

Predictability, Cache, and the
Architectures Underneath

SAMPAD ACHARYA



20
26



Writing Low-Latency C++: Predictability, Cache, and the Architectures Underneath

Engineering

Bloomberg

CppCon 2026
September 17, 2026

Sampad Acharya
Senior Software Engineer, Fixed Income
Trading

TechAtBloomberg.com

Agenda

- Introduction
- Mental Models
- STL Containers
- Memory Layout & Cache Behavior
- Atomics vs. Mutexes
- Threading & Scheduling
- Benchmarking & Observability
- Conclusion

Frequently Used Terms

TLB	Translation Lookaside Buffer
SIMD	Single Instruction, Multiple Data
LL/SC	Load-Linked / Store-Conditional
LSE	Large System Extensions
Tail Latency	the slow end of a latency distribution — the p99, p999, or p9999 requests

Agenda

- **Introduction**
- Mental Models
- STL Containers
- Memory Layout & Cache Behavior
- Atomics vs. Mutexes
- Threading & Scheduling
- Benchmarking & Observability
- Conclusion

Why this talk exists

- In low-latency systems, correctness is table stakes
- What separates good from great is predictability
- Latency is a feature, and if you don't design for it explicitly, you lose it accidentally
- I love benchmarking

Who this talk is for

- This talk is for anyone building low latency systems or engines, robotics, real-time infra, or any system where tail latency determines success
- If you've ever wondered why your code is 'fast' but still misses deadlines
- If you love benchmarking

Agenda

- Introduction
- **Mental Models**
- STL Containers
- Memory Layout & Cache Behavior
- Atomics vs. Mutexes
- Threading & Scheduling
- Benchmarking & Observability
- Conclusion

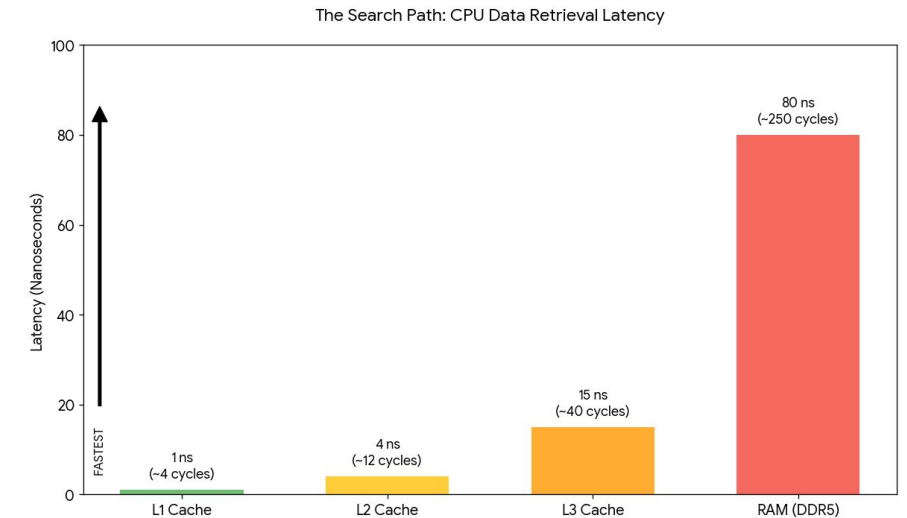
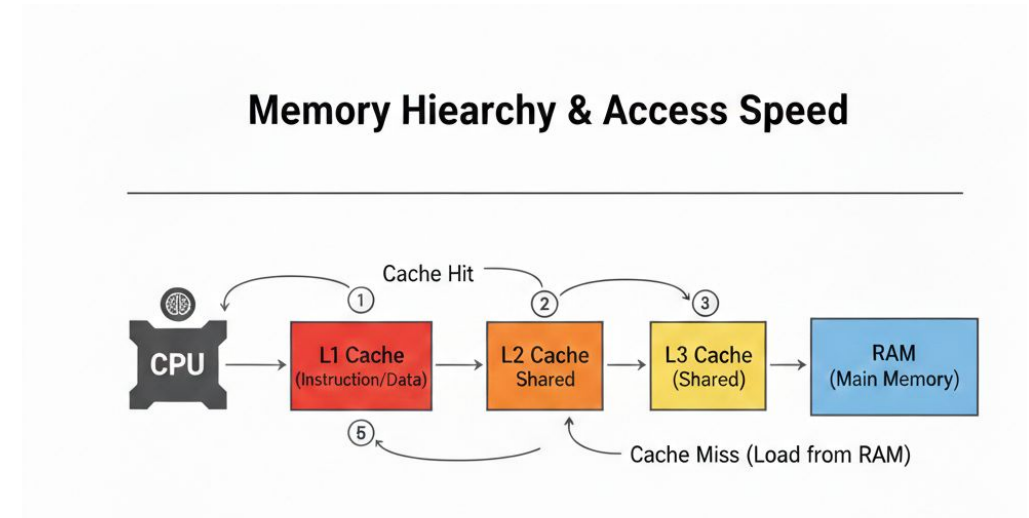
CPU-Centric Thinking

- The CPU doesn't care about your abstractions
- It cares about cache lines, branches, and memory dependencies
- Low-latency engineering is about aligning your code with what the CPU actually does

The Latency Stack

- Every time you miss L1, you pay
- Miss L2, you pay more
- Miss L3, you're in trouble
- Hit RAM, and your latency budget is gone!

Understanding this stack is the foundation of everything else



Predictability > Raw Speed

- A system that is 10% slower but predictable will outperform a system that is 10% faster but jittery
- Tail latency dominates real workloads

Avoiding Accidental Complexity

- Allocations and cache misses are the silent killers
- Branch predictions
- Out-of-Order execution can help a lot with the latency

Why A “Fast(er) Average” System Can Still Lose

- **Experiment Setup**
- Run **1M samples** of two systems:
 - **Predictable system:** Always does the same amount of CPU work
 - **Jittery system:** Usually does less work, but hits a **massive spike 1% of the time**
- Measure latency in nanoseconds
- Compute **mean, p50, p90, p99, p999**

Experiment Setup

```
// CPU-bound work
double do_work(int iterations) {
    double x = 0.0;
    for (int i = 0; i < iterations; ++i) {
        x += std::sin(i * 0.001);
    }
    return x;
}
```

```
// predictable always does this much work
const int predictable_iters = 120;

// jittery usually does this much work
const int jitter_base_iters = 100;

// rare spike adds this much work
const int jitter_extra_iters = 1000;
```

Sample Size = 1 M

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Experiment Setup

Two ways

Jittery loop:

- 99% time does 100 iters of work
- 1% iterations spike to 1100 iters (11×)

Predictable loop:

- Always does 120 iters
- No branches, no surprises

Results

Predictable system (ns):

mean : 219.172
p50 : 208
p90 : 250
p99 : 250
p999 : 416

Jittery system (ns):

mean : 224.688
p50 : 208
p90 : 250
p99 : 2250
p999 : 3166

- **Predictable:** $1\text{M} \times 120 = \mathbf{120\text{M}}$ ops
- **Jittery:** $999\text{k} \times 100 + 1\text{k} \times 11000 = \mathbf{110\text{M}}$ ops

Predictable path does more total work, but still has lower average

Even though the averages are the same for both cases, p99/p999 show the real picture

Observations

- Average lies for CPU-bound operations
- Even 1% jitter can degrade your overall performance if not tackled properly
- Carefully observe P95, P99, P999

The Three Rules of Low-Latency C++

- Minimize **work**
- Minimize **unpredictability**
- Minimize **memory movement**

Everything else is a corollary

Agenda

- Introduction
- Mental Models
- **STL Containers**
- Memory Layout & Cache Behavior
- Atomics vs. Mutexes
- Threading & Scheduling
- Benchmarking & Observability
- Conclusion

Containers as Latency Contracts

- When you choose a container, you're choosing a performance profile
- Most latency bugs start with the wrong container

`std::vector` and *reserve()*

- **reserve(N)** allocates raw storage sufficient for at least **N * sizeof(T) bytes** (plus allocator overhead)
- It **does not** construct any objects
- **No dynamic reallocation**

Other container with same behavior as vector:

- `std::string`, `std::basic_string`
- `std::unordered_map`, `std::unordered_set`
 - No `reserve()` on the container itself, but they have `reserve()` for bucket count
 - `reserve(n) = rehash(ceil(n / max_load_factor()))`

Call	<code>max_load_factor</code>	Resulting buckets
<code>reserve(1000)</code>	1.0	~1000
<code>reserve(1000)</code>	0.5	~2000

With *reserve()* vs. without *reserve()*

```
struct BigObject {  
    // 1 KB object to amplify reallocation cost  
    char data[1024];  
};
```

```
std::vector<BigObject> predictable;  
predictable.reserve(N);
```

```
std::vector<BigObject> jittery; // no reserve()
```

```
// predictable path  
for (int i = 0; i < N; ++i) {  
    auto start = Clock::now();  
    predictable.push_back(BigObject{});  
    auto end = Clock::now();  
    predictable_lat.push_back(std::chrono::duration_cast<ns>(end - start).count());  
}  
  
// jittery path  
for (int i = 0; i < N; ++i) {  
    auto start = Clock::now();  
    jittery.push_back(BigObject{});  
    auto end = Clock::now();  
    jittery_lat.push_back(std::chrono::duration_cast<ns>(end - start).count());  
}
```

reserve() and experimental data

Vector Size	Avg without reserve	Avg with reserve	Speedup
100	2.59 μ s	0.13 μ s	20.7×
100,000	139.7 μ s	85.3 μ s	1.6×
100,000,000	254.6 ms	106.2 ms	2.4×

reserve() and experimental data

$$S(n) = \frac{\alpha \cdot \lceil \log_2 n \rceil + 2\beta n}{\alpha + \beta n}$$

α = cost per allocation
 β = cost per element

small n : $S \rightarrow \lceil \log_2 n \rceil$ "counting mallocs"
large n : $S \rightarrow 2$ "counting bytes"

Observations

- Average is important for operations with amortized cost differences
- Dynamic allocation is extremely dangerous
- Use the power of *reserve()* where you can

Finding Jitters

- Tail latency matters when the spikes are too large to dominate the tail
- Average is important for operations with amortized cost differences where the spikes are too small

Benchmark	Spike Cause	Spike Size	Spike Frequency	Best Metric
CPU jitter	Rare huge extra work	Very large	~1%	p99 / p999
Vector reallocation	Rare reallocations	Moderate	≪1%	mean

Don't be afraid to write your own containers

- As a designer, you must know what is best for your architecture
- STLs are general purpose; you can write better containers for your specific cases
- Understand how the container works behind the scenes; it will help you design better

Agenda

- Introduction
- Mental Models
- STL Containers
- **Memory Layout & Cache Behavior**
- Atomics vs. Mutexes
- Threading & Scheduling
- Benchmarking & Observability
- Conclusion

CPU Caches: The Real Battlefield

- Most latency problems are memory problems
- If you fix memory movement, you fix latency

Struct Layout and what matters?

- Padding and alignment matter
- A poorly aligned struct can double your memory footprint and halve your performance
- Highest-to-Lowest size alignment

Array of Structures (AoS) vs. Structure of Arrays (SoA)

- Structure of Arrays wins when you access fields **independently**
- Array of Structures wins when you access objects **holistically**

Demo

```
struct DateOfBirth {  
    int year;  
    int month;  
    int day;  
};
```

```
struct Person {  
    string name;  
    int age;  
    DateOfBirth dob;  
    //char address[128];  
};
```

```
// ----- SoA Layout -----
```

```
struct PeopleSoA {  
    vector<string> names;  
    vector<int> ages;  
    vector<DateOfBirth> dobs;
```

```
    PeopleSoA(size_t n) {  
        names.reserve(n);  
        ages.reserve(n);  
        dobs.reserve(n);  
    }
```

```
};
```

Demo

Goal:

- Show how data layout affects CPU efficiency by benchmarking two ways of storing the same logical data:
 - AoS: Each *Person* is a full object with name, age, and date of birth
 - SoA: Each field is stored in its own contiguous array
- The benchmark computes the **average age across 2 million people**, repeated 10 times, and compares the timing

Person Struct Memory Alignment

```
struct DateOfBirth {  
    int year;    // 4 bytes  
    int month;   // 4 bytes  
    int day;     // 4 bytes  
};
```

```
struct Person {  
    string name;    // 24 or 32 bytes  
    int age;        // 4 bytes  
    DateOfBirth dob; // 12 bytes  
};
```

Why This Matters

Modern CPUs love:

- Contiguous memory
- Predictable access patterns
- Minimal cache waste

AoS and SoA behave very differently here:

- **AoS:** Iterating over `people[i].age` still loads the *entire struct* (name, DOB, etc.) into cache
- **SoA:** Iterating over `ages[i]` touches only the int array—tight, compact, cache-friendly

This demo shows how layout alone can change performance

Magic happens on x86 system

Build	AoS	SoA	Speedup
-O2 -march=native	3.60–3.78 ms	0.48–0.52 ms	~7×
-O3 -march=native	3.10 ms	0.19 ms	~16×

Person Struct Memory Alignment with “address”

```
struct DateOfBirth {  
    int year;  
    int month;  
    int day;  
};                                // sizeof = 12, alignof = 4  
  
struct Person {  
    std::string name;           // 32 B  
    int age;                   // 4 B  
    DateOfBirth dob;           // 12 B  
    char address[128];  
};                                // sizeof = 176, alignof = 8
```

Cache Line 1 (0–64 B)

[name (32B)][age (4B)][dob (12B)][address[0:16] (16B)]

Cache Line 2 (64–128 B)

[address[16:80] (64B)]

Cache Line 3 (128–176 B)

[address[80:128] (48B)]

Person Struct Memory Alignment with “address”

Cache Line Visualization

```
struct DateOfBirth {  
    int year;  
    int month;  
    int day;  
}; // sizeof = 12, alignof = 4  
  
struct Person {  
    std::string name; // 32 B  
    int age; // 4 B  
    DateOfBirth dob; // 12 B  
    char address[128];  
}; // sizeof = 176, alignof = 8
```

- Since a standard cache line is **64 bytes**
- Person struct spans nearly 3 - 4 cache lines (176 bytes over 64-byte cache lines)
- 32 KB L1 cache has $(32 * 1024) / 64 =$ **512 cache lines**
- Can fit only $(32 * 1024) / 176 =$ **186.18 → 186 structs**

Observations

- Architecture matters, know your system and environment
- SoA is better in similar cases showed in example
- Separation of data is not bad

Hot vs. Cold Data Separation

```
struct DateOfBirth {
    int year;    // 4 bytes
    int month;   // 4 bytes
    int day;     // 4 bytes
    // Total: 12 bytes
};

struct PersonCold {
    char address[128]; // 128 bytes
    // Total: 128 bytes
};

struct PersonHot {
    string name;        // 24 bytes on GCC/Clang, 32 bytes on MSVC
    int age;             // 4 bytes
    DateOfBirth dob;     // 12 bytes
    PersonCold* cold;    // 8 bytes (pointer on 64-bit)
    // Total:
    // GCC/Clang: 24 + 4 + 12 + 8 = 48 bytes
    // MSVC:      32 + 4 + 12 + 8 = 56 bytes
};
```

- Split structs so hot fields stay in cache and cold fields stay out
- This reduces cache pollution

Results

Build	AoS (168 B)	Hot and Cold path	Speedup
-O2 -march=native	7.85 ms	6.33 ms	1.24×
-O3 -march=native	5.73 ms	4.05 ms	1.41×

Which one is better?

```
struct TestAligned_A{  
    char    flag;  
    double  value;  
    char    tag;  
    int     id;  
    short   code;  
    double  amount;  
    char    category;  
};
```

OR

```
struct TestAligned_B {  
    double  value;  
    double  amount;  
    int     id;  
    short   code;  
    char    flag;  
    char    tag;  
    char    category;  
};
```

Quick experiment

Set up:

- 4,000,000 records
- Sum .amount
- Core used: 2

Build	Run #1 (Misaligned/ Aligned)	Run #2	Run #3
-O2 -march=native	1.45x	1.42x	1.41x
-O3 -march=native	1.41x	1.39x	1.40x

Misalignment

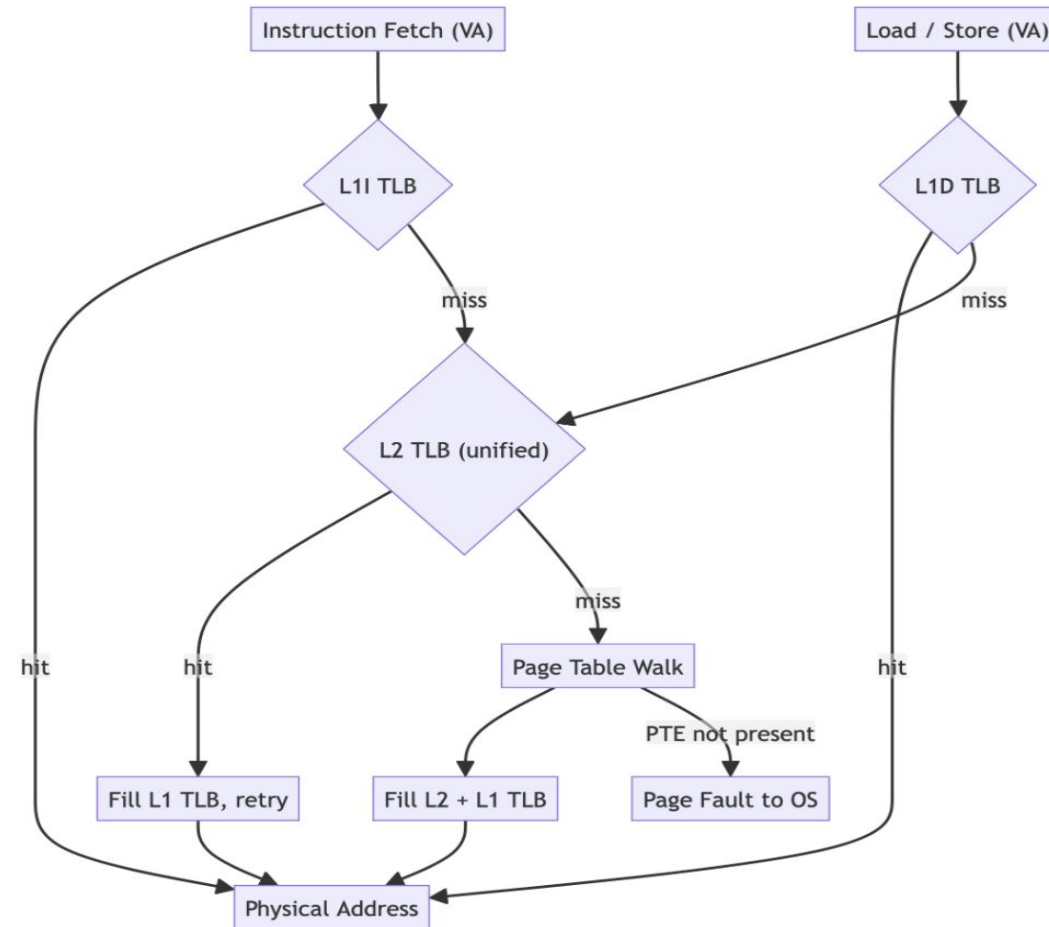
```
struct TestAligned_A{  
    char    flag;        // 1    align 1    offset 0  
    double  value;       // 8    align 8    offset 8    (7 B pad before it)  
    char    tag;         // 1    align 1    offset 16  
    int     id;          // 4    align 4    offset 20    (3 B pad before it)  
    short   code;        // 2    align 2    offset 24  
    double  amount;      // 8    align 8    offset 32    (6 B pad before it)  
    char    category;    // 1    align 1    offset 40  
};                        // sizeof = 48    (17 B wasted of 31 B of data)
```

Aligned

```
struct TestAligned_B {  
    double value;    // 8    align 8    offset 0  
    double amount;  // 8    align 8    offset 8  
    int    id;       // 4    align 4    offset 16  
    short  code;     // 2    align 2    offset 20  
    char   flag;     // 1    align 1    offset 22  
    char   tag;      // 1    align 1    offset 23  
    char   category; // 1    align 1    offset 24  
};                  // sizeof = 32 (unavoidable 7 B tail pad only)
```

Translation Lookaside Buffer (TLB) layout

- L1 TLB != L1
- L1I TLB != L1I (I = Instruction)
- L1D TLB != L1D (D = Data)
- TLBs sits directly inside the CPU's Memory Management Unit (MMU)



Does the hot and cold path helps with TLB misses?

Set up:

- `constexpr size_t N = 2'000'000`
- `constexpr int REPS = 60`
- Both print the same checksum (5,340,000,000)
- Identical data, only the layout differs

Measurements:

- dTLB-loads
- dTLB-load-misses
- page-faults
- cache-references
- cache-misses
- instructions
- cycles

Results

Metric	AoS (176 B)	Hot/Cold (56 B)	Ratio
dTLB-load-misses	195,992	89,910	2.18×
dTLB-loads (total)	484 M	490 M	~1×
dTLB-miss rate	0.04%	0.02%	2×
cache-misses	168 M	81.4 M	2.06×
cache-references	183 M	118 M	1.55×
cache-miss rate	91.8%	68.9%	—
instructions	2.11 B	2.13 B	~1×
cycles	1.75 B	1.71 B	~1×
page-faults	86,067	89,975	~1×
wall time	0.930 s	0.633 s	1.47×

Any memory effects on SIMD performances?

Set up:

- Same dot product: $sum += a[i] * b[i]$
- 3 implementations, only accumulator count/vectorization differs:
 - 4-accumulator
 - 1-accumulator
 - Scalar
- Tested in 2 regimes:
 - L1-resident (32 KB, fits cache)
 - Memory-bound (128 MB, far exceeds L3)

Any memory effects on SIMD performances?

Scalar:

```
float dot(const float* a, const float* b, size_t n) {  
    float sum = 0.0f;  
    for (size_t i = 0; i < n; i++) sum += a[i] * b[i];  
    return sum;  
}
```

Any memory effects on SIMD performances?

1 Accumulator

```
for (size_t i = 0; i < n; i += 8)
    acc = _mm256_fmadd_ps(_mm256_loadu_ps(a+i), _mm256_loadu_ps(b+i), acc);
float t[8]; _mm256_storeu_ps(t, acc);
return t[0]+t[1]+t[2]+t[3]+t[4]+t[5]+t[6]+t[7];
```

4 Accumulator

```
for (size_t i = 0; i < n; i += 32) {
    acc0=_mm256_fmadd_ps(_mm256_loadu_ps(a+i), _mm256_loadu_ps(b+i), acc0);
    acc1=_mm256_fmadd_ps(_mm256_loadu_ps(a+i+8), _mm256_loadu_ps(b+i+8), acc1);
    acc2=_mm256_fmadd_ps(_mm256_loadu_ps(a+i+16), _mm256_loadu_ps(b+i+16), acc2);
    acc3=_mm256_fmadd_ps(_mm256_loadu_ps(a+i+24), _mm256_loadu_ps(b+i+24), acc3);
}
__m256 acc=_mm256_add_ps(_mm256_add_ps(acc0,acc1),_mm256_add_ps(acc2,acc3));
float t[8]; _mm256_storeu_ps(t, acc);
return t[0]+t[1]+t[2]+t[3]+t[4]+t[5]+t[6]+t[7];
```

Results

Regime	Variant	time (ms)	cycles	instructions	LLC-miss %
L1	4acc	0.00009	50.4M	157.1M	9.98%
L1	1acc	0.00033	175.8M	259.1M	9.29%
L1	scalar	0.00156	813.0M	2,461.1M	9.66%
Memory	4acc	4.548	808.2M	820.0M	46.99%
Memory	1acc	4.636	813.5M	902.3M	48.75%
Memory	scalar	8.103	1,172.5M	2,706.4M	66.54%

Results

Comparison	L1-resident	Memory-bound
4acc vs 1acc	3.70×	1.02× (tied)
1acc vs scalar	4.68×	1.75×
4acc vs scalar	17.33×	1.78×

Results: Why 4 acc and 1 acc tied for memory bound

Metric	4acc	1acc	Ratio (4/1)
time (ms)	8.65	8.97	0.964
L1-dcache-loads	118.3M	118.0M	1.003
L1-dcache-load-misses	35.72M	37.79M	0.945
LLC-loads	23.47M	24.90M	0.942
LLC-load-misses	12.25M	15.14M	0.809
l2_lines_in.all	44.59M	44.55M	1.001
mem_load_retired.l3_hit	2.32M	1.64M	1.415
mem_load_retired.l3_miss	1.70M	0.99M	1.716
longest_lat_cache.reference	44.06M	43.88M	1.004
longest_lat_cache.miss	26.67M	26.44M	1.009

Memory layout checklist

- Keep hot and cold fields separate
- Group frequently accessed fields together
- Avoid unnecessary padding by ordering fields from largest to smallest
- Ensure hot data fits within as few cache lines as possible
- Keep working sets small enough to fit in L1/L2

Agenda

- Introduction
- Mental Models
- STL Containers
- Memory Layout & Cache Behavior
- **Atomics vs. Mutexes**
- Threading & Scheduling
- Benchmarking & Observability
- Conclusion

Who wins: atomic or mutex?

- **It depends** — hardware, contention, memory ordering, and instruction semantics

Different Atomic Operations

- **fetch-add**
- **compare-exchange (CAS)**
- **test-and-set**

Memory orderings: relaxed, acquire, release, acq_rel, seq_cst

Case Study Steps

Multithreaded approach to increment a counter:

- Number of Threads: 4/10/18/24/32/40/400
- Number of Iterations: 100,000

Different mechanisms:

- Mutex
- Atomic fetch_add relaxed
- Atomic fetch_add seq_cst
- Atomic weak CAS loop with memory_order_relaxed
- Atomic weak CAS loop with memory_order_seq_cst

Case Study Results

```
Target Architecture: ARM64  
Threads: 4 | Iters: 100000
```

```
-----  
Standard Mutex           : 19.5766 ms  
Atomic fetch_add (Relaxed) : 7.50338 ms  
Atomic fetch_add (Seq_Cst) : 10.5273 ms  
Atomic CAS (Relaxed)      : 4.99821 ms  
Atomic CAS (Seq_Cst)     : 23.6881 ms
```

Case Study Results

Target Architecture: ARM64
Threads: 10 | Iters: 100000

Standard Mutex	: 38.5822 ms
Atomic fetch_add (Relaxed)	: 44.4347 ms
Atomic fetch_add (Seq_Cst)	: 57.2459 ms
Atomic CAS (Relaxed)	: 265.73 ms
Atomic CAS (Seq_Cst)	: 304.635 ms

Target Architecture: ARM64
Threads: 24 | Iters: 100000

Standard Mutex	: 68.1654 ms
Atomic fetch_add (Relaxed)	: 101.715 ms
Atomic fetch_add (Seq_Cst)	: 140.412 ms
Atomic CAS (Relaxed)	: 861.901 ms
Atomic CAS (Seq_Cst)	: 845.104 ms

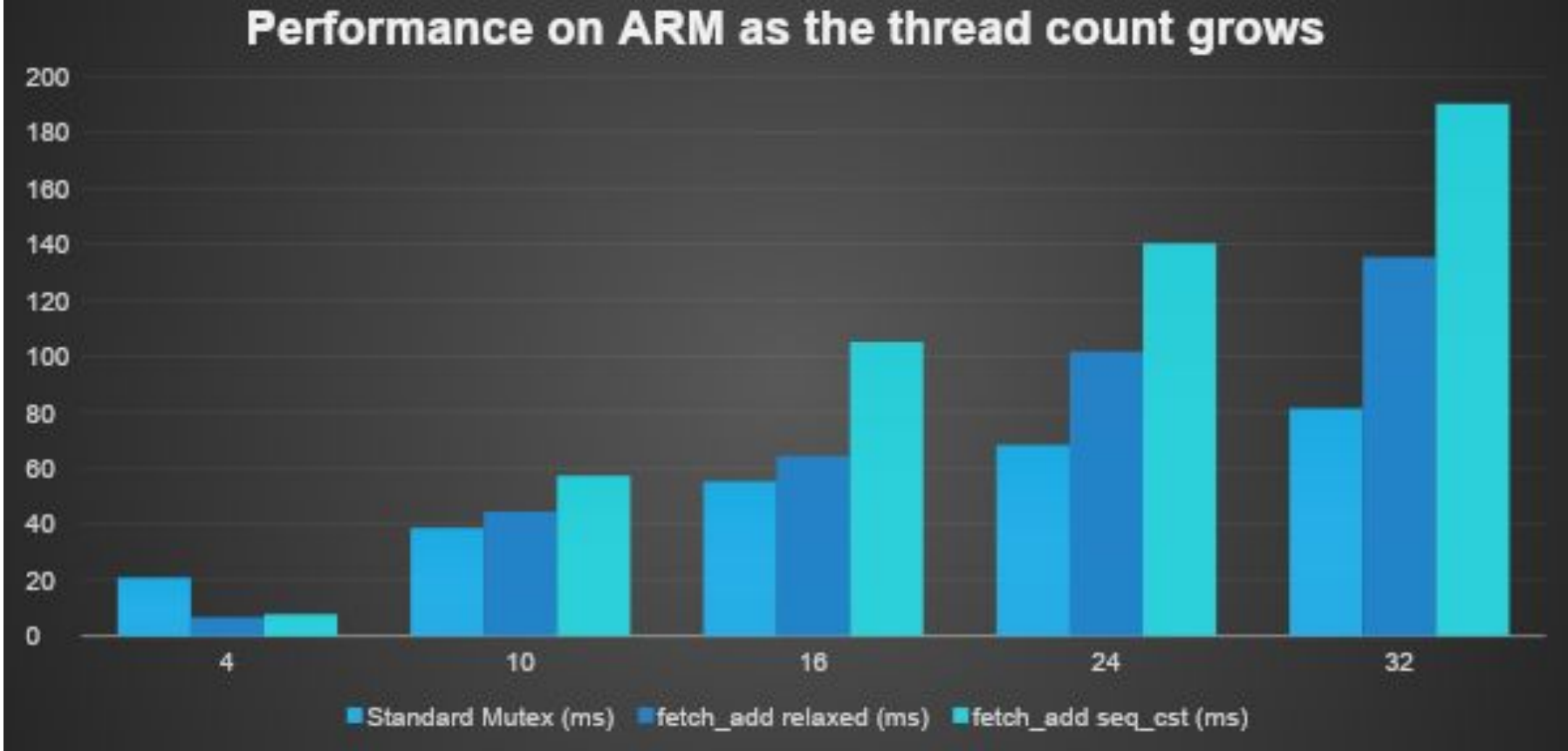
Target Architecture: ARM64
Threads: 16 | Iters: 100000

Standard Mutex	: 55.186 ms
Atomic fetch_add (Relaxed)	: 64.2943 ms
Atomic fetch_add (Seq_Cst)	: 105.066 ms
Atomic CAS (Relaxed)	: 561.839 ms
Atomic CAS (Seq_Cst)	: 620.098 ms

Target Architecture: ARM64
Threads: 32 | Iters: 100000

Standard Mutex	: 81.2704 ms
Atomic fetch_add (Relaxed)	: 135.424 ms
Atomic fetch_add (Seq_Cst)	: 190.183 ms
Atomic CAS (Relaxed)	: 1153.12 ms
Atomic CAS (Seq_Cst)	: 1161.43 ms

Performances on ARM as contention grows



Case Study Results of Extreme Cases

Threads	Mutex (ms)	fetch_add relaxed (ms)	fetch_add seq_cst (ms)	relaxed over mutex	seq_cst over mutex
4	19.5766	7.50338	10.5273	0.38x	0.54x
400	778.276	1850.29	2421.63	2.38x	3.11x
4000	8859.84	16451.9	25311.5	1.86x	2.86x

Case Study Results – x86

Multithreaded approach to increment a counter:

- Number of Threads: **40 / 4000**
- Number of Iterations: **100,000**

```
Target Architecture: x86_64
Threads: 40 | ITERS: 100000
-----
Standard Mutex           : 394.626 ms
Atomic fetch_add (Relaxed) : 97.7808 ms
Atomic fetch_add (Seq_Cst) : 96.1961 ms
Atomic CAS (Relaxed)      : 383.736 ms
Atomic CAS (Seq_Cst)     : 401.285 ms
```

```
Target Architecture: x86_64
Threads: 4000 | ITERS: 100000
-----
Standard Mutex           : 45078.1 ms
Atomic fetch_add (Relaxed) : 6735.84 ms
Atomic fetch_add (Seq_Cst) : 6725.27 ms
Atomic CAS (Relaxed)      : 35261.2 ms
Atomic CAS (Seq_Cst)     : 36384.6 ms
```

Observations

ARM:

- Mutex wins over Atomic under high contention
- Atomic wins over Mutex under low contention

X86:

- Atomic wins over Mutex under both low and high contentions

Explanation

- Every attempt requires a full memory barrier on ARM
- LL/SC* adds retry logic unlike x86 where it is tackled via pipeline stall
- Cache line ping-pong

(If you are using older ARM versions, before 8.1)

- LL/SC: Load-Linked / Store-Conditional
- New LSE (Large System Extensions) — replaces LL/SC retry loops with single-instruction atomics — no reservation, no software retry.

Full explanation

When Atomics Lose - Architecture-Dependent Performance of C++ Synchronization - Sampad Acharya



TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Takeaway

- Depending on architecture and contention, Atomic can lose to Mutex
- Depending on the memory order, like relaxed or seq_cst, the performance can be degraded further
- Contention makes everything worse

Atomic Checklist

- x86 workloads
- Low contention on ARM
- Simple counters
- Weak memory ordering acceptable

Mutex Checklist

- High contention
- Long critical sections
- ARM servers
- Need blocking instead of spinning

Agenda

- Introduction
- Mental Models
- STL Containers
- Memory Layout & Cache Behavior
- Atomics vs. Mutexes
- **Threading & Scheduling**
- Benchmarking & Observability
- Conclusion

Threads as a Latency Hazard

- Threads introduce context switches, migrations, and preemption
- They're powerful, but dangerous

Thread Pool demo on Unix kernel

- Fixed Size Thread Pool vs. Dynamic Size Thread Pool
- Maximum number of Threads for each side is 12

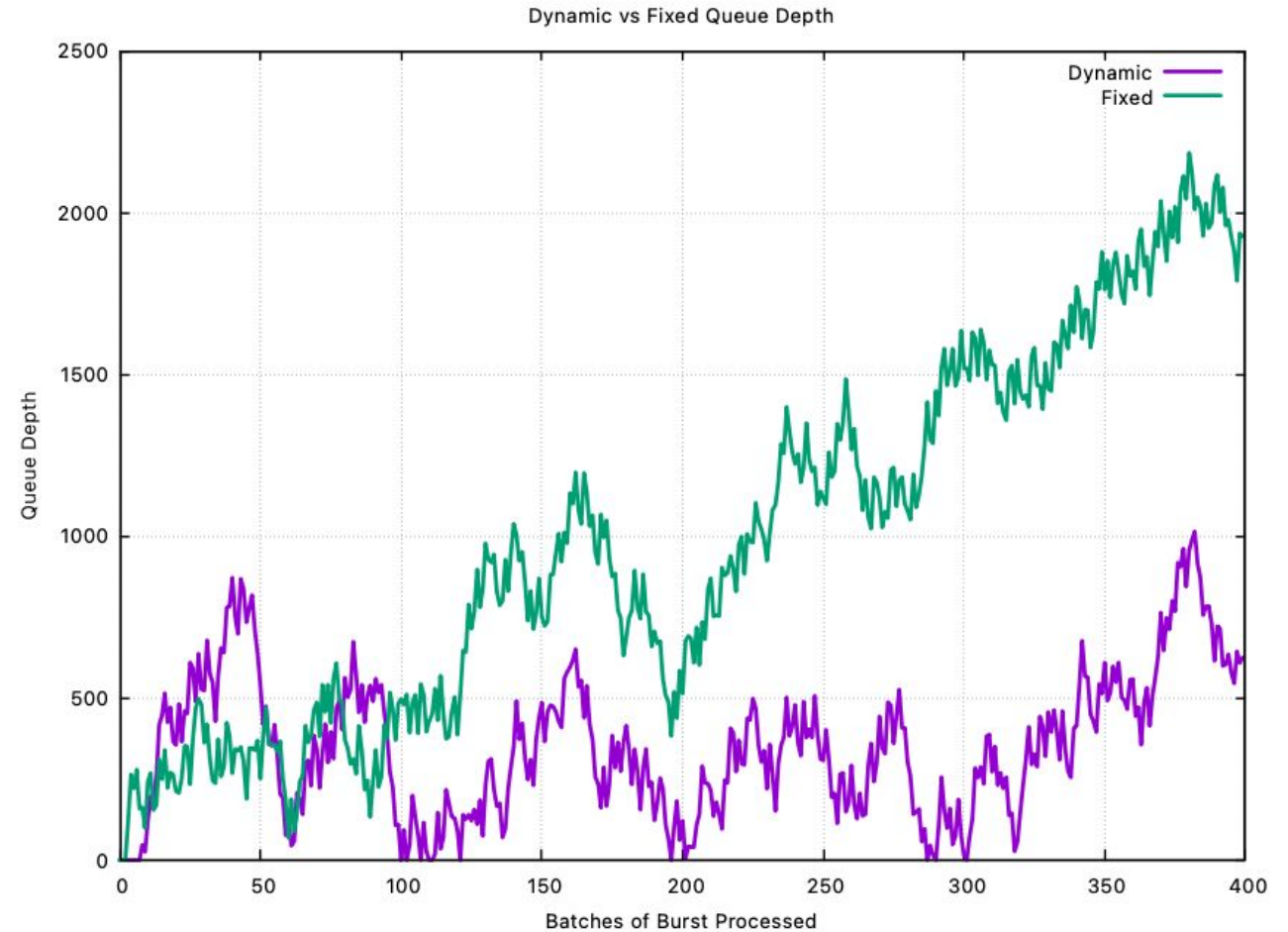
Fixed Size Thread Pool: It will start from 12 threads and continue with them all

Dynamic Size Thread Pool: It will start with 2 threads and, depending on the wait time for each task on the queue, it will scale up to maximum 12 threads, and then continue with 12 threads

Thread Pool demo: Result

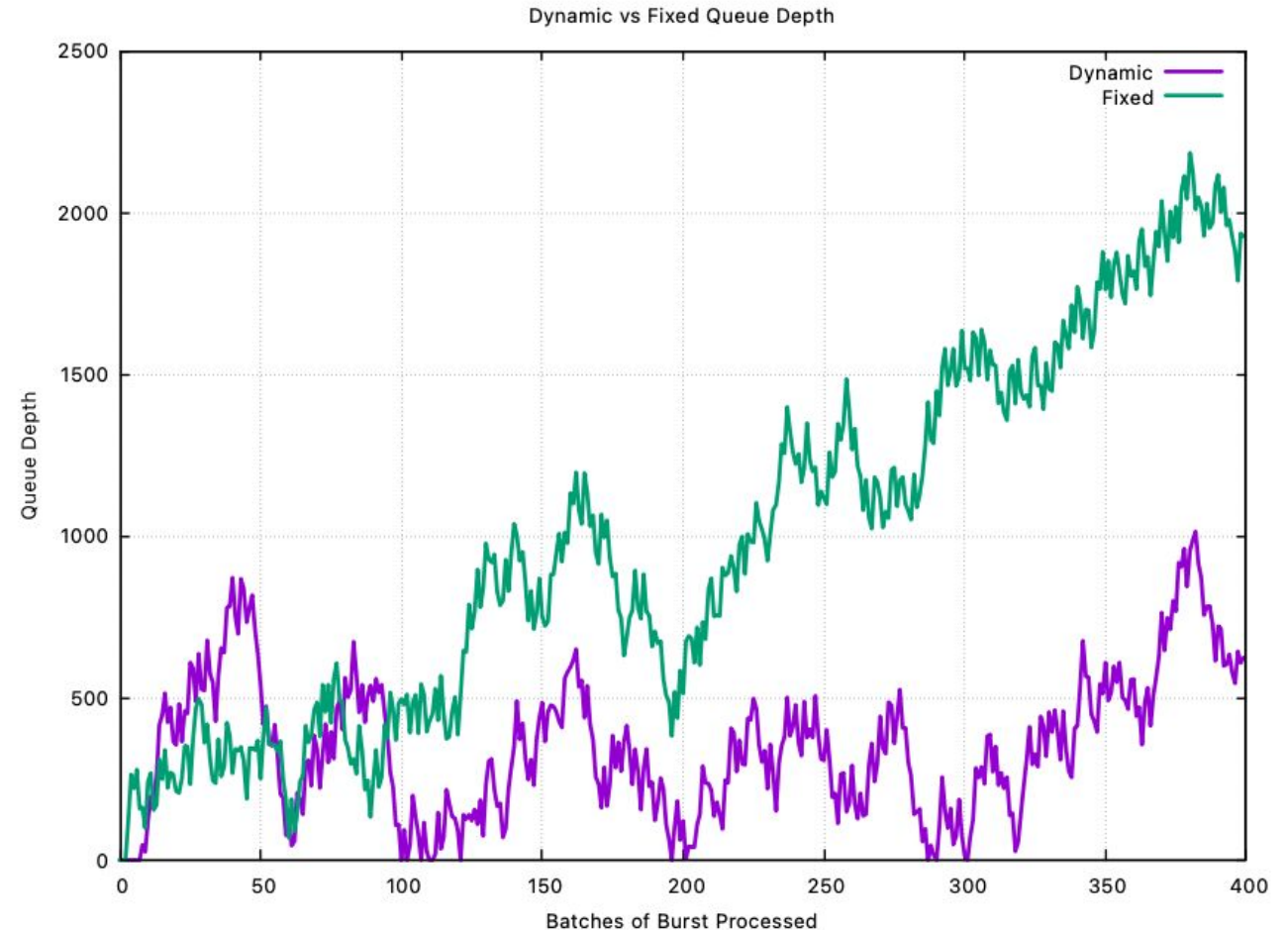
The Surprising Culprit: Worker Starvation in the Fixed Pool

- The dynamic pool had a **monitor thread** that checked queue wait times periodically
- The fixed pool had **no such thread**
- This small action breaks a starvation loop into which the fixed pool falls



Worker Starvation

- In the fixed pool, all 12 worker threads sit on the same condition variable and the same mutex
- **Under burst load, they form a tight lock-handoff cycle** in which the same worker thread keeps getting woken up and reacquiring the mutex
- This is where the intuition breaks down



More on Threading & Scheduling

- This experience will vary depending on OS and architecture
- More on this come in my next talk
- I will create a detailed experiments on this and explain the results

“Why would that delay execution? The worker is right there ready to take the lock.”

- The delay doesn't come from “not having a worker available.”
- The delay comes from how condition variables and mutexes actually behave under contention

There is **no** guarantee that:

- the longest-waiting thread wakes first
- threads wake in order
- a thread that just finished work goes to the back of the line
- all threads get a fair chance to acquire the mutex

The OS scheduler decides who wakes up, and it **does not** pick workers from a **FIFO**-like queue

How does the OS scheduler decide who wakes up?

The OS scheduler decides who wakes, and it tends to pick:

- the thread that was running most recently
- the thread that is still “hot” in the CPU scheduler
- the thread with the best cache locality

This means the same worker can repeatedly win the race to reacquire the mutex, forming a loop

If you have 4 threads A, B, C, D

OS will pick threads like below:

$A \rightarrow A \rightarrow A \rightarrow A \rightarrow A$

Meanwhile:

- B, C, D... never get scheduled
- the queue grows
- latency increases, throughput collapses

Even though “*A is doing work,*” the system becomes effectively single-threaded

Observations

- Know how environment (OS) works
- Concurrency is simple if you understand the whole process

Agenda

- Introduction
- Mental Models
- STL Containers
- Memory Layout & Cache Behavior
- Atomics vs. Mutexes
- Threading & Scheduling
- **Benchmarking & Observability**
- Conclusion

Avoiding Measurement Bias and Jitter

- Branch predictor poisoning
- Cache warming
- Compiler optimizations can all distort results

Benchmark dependencies

- Performance benchmarks depend on system characteristics
- Performance results vary dramatically based on:
 - CPU architecture
 - NUMA topology
 - Memory bandwidth & cache hierarchy
 - OS scheduler
 - Kernel version
 - Compiler optimizations
 - Microarchitectural features (prefetchers, store buffers, coherence protocol)

Latency Histograms

- Know your tools and the limitations
- Averages lie for CPU bound operations
- Percentiles tell the truth
- Always look at the distribution

Reproducibility

- Document compiler flags, CPU model, kernel version, and environment
- Without this, benchmarks are meaningless
- Benchmark results are system-dependent. They do not generalize globally, only within the context of the measured hardware and environment

Agenda

- Introduction
- Mental Models
- STL Containers
- Memory Layout & Cache Behavior
- Atomics vs. Mutexes
- Threading & Scheduling
- Benchmarking & Observability
- **Conclusion**

Conclusion

- Think in cache
- Use the power of containers but beware of pitfalls
- Design for predictability
- Know your architecture and environment
- Benchmark everything
- Absolute benchmark is a myth

Bloomberg

Engineering

Thank you!

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Questions?

Interested in C++ @ Bloomberg?

We are hiring: bloomberg.com/engineering

Career Opportunities



Learn more about our work



Engineering

Bloomberg

TechAtBloomberg.com